

Browser Calling with Exotel

from zero to a live call

Account requirements, the complete API surface, exact request and response bodies, and the implementation details required for a working integration. Based on a full production build.

What this covers. Turning a web page into a working telephone: provisioning an Exotel application, creating a WebRTC agent, registering a browser onto Exotel's SIP network, placing and receiving calls, and recording them. Every API call shown here was executed against a live account, and every response body is a real one with secrets removed. The final section collects five implementation details that repay attention early, each with what you would observe, why it happens, and how to handle it.

DOCUMENT TYPE

Engineering guide and field report

PLATFORM

Exotel IP-PSTN WebRTC, Mumbai region

VERIFIED AGAINST

Account `exotel1243m`, August 2026

AUDIENCE

Engineers, with plain-language notes throughout

SDK

`@exotel-npm-dev/exotel-ip-calling-crm-websdk` 1.2.2

STATUS

Outbound calling confirmed live, 19 second test call

Contents

PART I • ORIENTATION

- 1 What browser calling is
- 2 System components and the role of the backend
- 3 Vocabulary

PART II • WHAT MUST BE TRUE AT EXOTEL

- 4 Account prerequisites
- 5 The four credential pairs
- 6 The two API families

PART III • PROVISIONING

- 7 Step one: mint a customer token
- 8 Step two: register the application
- 9 Step three: mint an application token
- 10 Step four: create the WebRTC agent
- 11 Step five: turn on recording
- 12 Step six: verification

PART IV • THE BROWSER

- 13 What the SDK does on initialisation
- 14 SIP registration over WebSocket
- 15 Placing an outbound call
- 16 Receiving an inbound call
- 17 Microphone access and browser policy

PART V • OPERATIONS

- 18 Call recording
- 19 Reading call detail records
- 20 Production deployment

PART VI • IMPLEMENTATION NOTES

- 21 Five implementation considerations
- 22 Error reference
- 23 Verification checklist

APPENDICES

- A Endpoint reference
- B Glossary

Orientation

System overview, the components involved, and the terminology used throughout this document.

1 What browser calling is

A normal phone call travels over the telephone network. A WebRTC call travels over the internet. Exotel's IP-PSTN product is the bridge between the two, and that hyphenated name is the whole idea: IP on one side, the public switched telephone network on the other.

Your agent sits in front of a browser with a headset. When they call a customer's mobile, the audio leaves the browser as internet packets, reaches Exotel, and Exotel converts it into an ordinary phone call that arrives on the customer's handset. The customer sees your ExoPhone number and has no idea a browser was involved. In the other direction, a customer dials your ExoPhone, Exotel converts that call into internet packets, and the agent's browser rings.

IN PLAIN LANGUAGE

Exotel acts as a translator between two networks. The browser sends audio over the internet; a mobile phone expects audio from the telephone network. Exotel converts between the two in real time, so neither side is aware of the other's format. For the agent this means a computer, a headset and a web page are sufficient. No desk phone, SIM card or telephone line is required.

Two things follow from this that shape everything else in the guide. First, because the browser is doing real-time audio, the browser's own rules apply: microphone permission, secure origins, autoplay policy. Second, because Exotel is placing real phone calls on your behalf, Exotel's rules apply: account verification, balance, and a provisioned identity for every agent. A substantial part of the integration effort is meeting these two sets of requirements rather than writing application code.

2 System components and the role of the backend

Three components are involved. The distinction that matters is which of them is permitted to hold credentials.

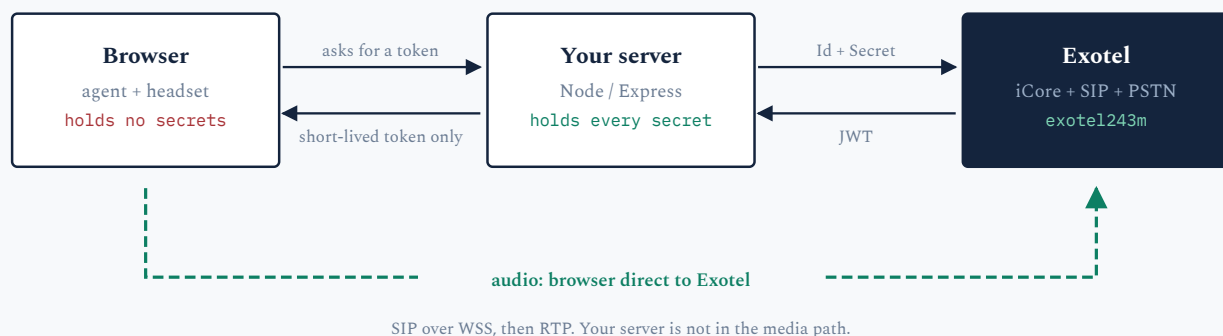


Figure 1. Control and media take different routes. Your server brokers credentials and never carries voice; the audio goes straight from the browser to Exotel. That is why a small server is enough, and why server latency does not affect call quality.

The reason a backend is mandatory rather than optional: the browser needs an access token to talk to Exotel, and that token is minted from a secret. If you put the secret in JavaScript, you have published it. Anyone reading your page source could place calls billed to your account. So the secret lives on your server, the server mints tokens, and the browser receives only the token.

3 Vocabulary

TERMS USED THROUGHOUT THIS DOCUMENT

Term	Meaning
PSTN	Public Switched Telephone Network. The ordinary phone system that mobiles and landlines live on.
WebRTC	The browser technology that carries live audio and video. Built into Chrome, Edge, Safari and Firefox.
SIP	Session Initiation Protocol. The signalling language that sets up, answers and ends calls. It does not carry the audio itself.
RTP	The protocol that does carry the audio, once SIP has agreed the call.
ExoPhone	Your Exotel virtual number. It is the caller ID customers see, and the number they ring to reach you.
iCore	Exotel's integrations service, hosted at <code>integrationscore.mum1.exotel.com</code> for the Mumbai region. All WebRTC provisioning lives here.
Application	An Exotel object representing your integration. It owns agents and settings, and has its own credential pair.
User mapping	The record that turns an agent's email address into a SIP identity. Creating one is what "provisioning an agent" means.
CallSid	Exotel's unique identifier for a call. You need it to look up status, duration and recordings.

Exotel account requirements

Account conditions that must be met before integration, and the credentials required for each API.

4 Account prerequisites

Four conditions must hold on the Exotel side before a browser can place a call. Each one failed at least once during this build, and each failure produces a different error, so checking them in this order will save time.

4.1 The account must not be on trial

A trial account can authenticate, can register an application, and can place click-to-call calls. It cannot create a WebRTC agent. The attempt returns a flat refusal:

RESPONSE CREATING AN AGENT ON A TRIAL ACCOUNT

```
{
  "Status": "Failed",
  "Code": 403,
  "Error": "This is a trial account. Operation not permitted."
}
```

There is no API that lifts this; it is enabled on Exotel's side. Ask your account manager, or email hello@exotel.in, to have the account enabled for IP-PSTN WebRTC, which normally means a signed agreement and completed KYC. Section 5.1 has a template that requests this and your customer credentials together, so one message covers both. You can confirm the state yourself through the Voice API, and the field to watch is `Type`:

GET ACCOUNT STATUS

```
GET https://api.in.exotel.com/v1/Accounts/exotel243m.json
Authorization: Basic base64(API_KEY:API_TOKEN)

// response, trimmed
{ "Account": {
  "Type": "Full",           // "Trial" blocks agent creation
  "Status": "active",
  "BillingType": "prepaid",
  "KycStatus": "completed"
} }
```

4.2 The account must have balance

On a prepaid account, calls are only placed while there is balance to bill them against. Everything else keeps working normally, so credentials, provisioning and SIP registration will all report healthy while calls are

declined. Keep an eye on the balance in the dashboard, and if calls are being declined while registration is green, check it first or ask Exotel support to confirm the account state.

4.3 You need an ExoPhone

The virtual number does two jobs. On outbound calls it is the caller ID the customer sees. On inbound calls it is the number customers dial. Each agent carries one on their record as `VirtualNumber`, and agents on the same team commonly share it. Examples in this guide use `08048332956`; substitute your own.

4.4 Inbound routing is configured in the dashboard

This step lives in the dashboard rather than the API, so it is easy to miss. Provisioning an agent gives that agent the ability to receive calls, but it does not tell Exotel to send any. What happens when someone dials your ExoPhone is decided by a call flow, and the flow is built in the Exotel dashboard, not through the APIs in this guide.

DASHBOARD STEP REQUIRED

In the Exotel dashboard, attach your ExoPhone to a flow whose applet connects the call to the VoIP or WebRTC agent, identified by the agent's email address. Until that exists, outbound browser calls work and inbound calls never arrive. Nothing in your code changes.

5 The four credential pairs

Exotel hands you several credentials with similar-sounding names, and using the wrong one produces an authorisation error that does not say which. There are four, and they are not interchangeable.

CREDENTIALS AND WHAT EACH IS FOR

Pair	Used for	Notes
API Key API Token	Voice API v1	Classic REST over HTTP Basic. Used for account status, call detail records, and click-to-call. Nothing to do with WebRTC.
Client ID Client Secret	Customer entity	Identifies you as the customer. Used once, to register an application. Your Client ID is also your CustomerID.
App ID App Secret	Application entity	Returned when you register the application. Everything WebRTC uses this pair. Store it.
SIP ID SIP Secret	The browser	Generated by Exotel per agent. You never handle it directly; the SDK fetches it. Never send it to a client you do not control.

5.1 Obtaining each credential

Two of the four pairs are obtained directly. The remaining two are issued by the platform during provisioning and are returned in API responses, so they will not be found in the dashboard.

HOW TO OBTAIN EACH CREDENTIAL

Pair	Where you get it
API Key API Token	Exotel dashboard, under the API settings for your account. Self-service.
Client ID Client Secret	Request from Exotel. These are customer-level WebRTC credentials and are not exposed in the dashboard. Email <code>hello@exotel.in</code> and ask for them, quoting your Account SID. See the template below.
App ID App Secret	Returned in the response when you register your application in section 8. Nothing to request.
SIP ID SIP Secret	Generated automatically when you create an agent in section 10. The SDK fetches them for you.

REQUEST THESE CREDENTIALS EARLY

The Client ID and Client Secret are the one dependency with a human in the loop, so send the email before you write any code. Everything in Parts I and II, plus reading the SDK and building your UI, can be done while you wait. The same message is a good place to confirm that your account is enabled for IP-PSTN WebRTC, which is the prerequisite in section 4.1, so you can resolve both in a single exchange.

EMAIL TEMPLATE · TO `HELLO@EXOTEL.IN`

```
To:      hello@exotel.in
Subject: WebRTC Client ID and Client Secret request | Account SID <your account sid>

Hello Exotel team,

We are integrating the Exotel IP-PSTN WebRTC WebSDK
(@exotel-npm-dev/exotel-ip-calling-crm-websdk) into our application, so that
our team can make and receive calls from the browser.

To register our application we need the customer level credentials:
  1. Client ID
  2. Client Secret

Please could you also confirm that this account is enabled for IP-PSTN WebRTC,
so that we can create WebRTC agents.

Account SID   : <your account sid, for example exotel243m>
Registered email : <the email on the Exotel account>
ExoPhone      : <your virtual number>
Region        : <for example Mumbai>
Use case      : <one line, for example browser calling for our support team>

Thank you,
<your name, company>
```

Include the Account SID in the subject line as well as the body. Support cannot look anything up without it, and putting it in the subject usually means the first reply already contains what you asked for rather than a request for more detail.

CONFIRM WHICH VALUE IS THE KEY AND WHICH IS THE TOKEN

The API Key and the API Token are both opaque strings of similar length and the same character set. Nothing in either value identifies which is which, so once they have been copied out of a portal, pasted into a spreadsheet, forwarded in an email or handed over by a colleague, the order is easy to lose. HTTP Basic auth is order-sensitive: the key is the username and the token is the password.

A reversed pair and an invalid credential produce an identical failure, so the error message does not distinguish between them. Confirm the order with a single read-only request, attempted both ways, before concluding that the credentials are invalid or that the account is misconfigured.

WHICH ORDER IS CORRECT?

```
// GET /v1/Accounts/<sid>.json is read only and has no side effects,
// which makes it the right probe. Whichever order returns 200 is correct.

const probe = (user, pass) =>
  fetch(`https://api.in.exotel.com/v1/Accounts/${sid}.json`, {
    headers: { Authorization: "Basic " + btoa(`${user}:${pass}`) }
  }).then(r => r.status);

await probe(A, B); // 200 means A is the key, B is the token
await probe(B, A); // 200 means they were labelled the wrong way round
// 401 or 403 both ways means the credentials really are wrong
```

This is worth doing on every new account, not only when something breaks. The Voice API is the first thing you will use, in section 4.1, and starting from a confirmed credential order removes an entire class of confusion from everything that follows. It happened on the account this guide was written against, and it is common enough to be the first thing to rule out.

6 The two API families

Exotel exposes two API families with separate base URLs and separate authentication schemes. Identifying which family an endpoint belongs to determines how it must be authenticated.

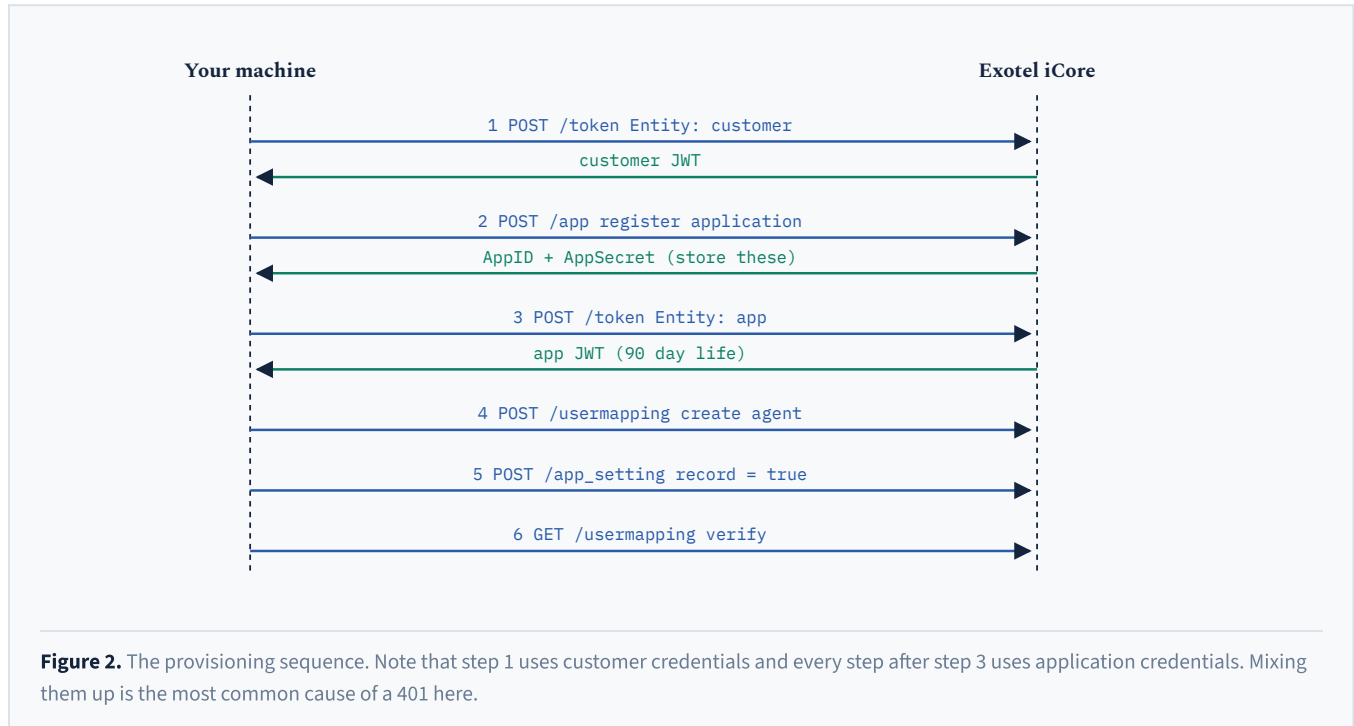
THE TWO FAMILIES

	iCore integrations	Voice API v1
Base URL	integrationscore.mum1.exotel.com	api.in.exotel.com
Auth	Bearer-style JWT in Authorization	HTTP Basic, key and token
Style	JSON in, JSON out	Form-encoded in, JSON out
Casing	PascalCase keys	PascalCase keys
Owns	Applications, agents, settings, WebRTC calls	Account status, call records, click-to-call
Region	mum1 for Mumbai	api.in for India

Everything in Part III is iCore. The Voice API appears only twice in this guide: to check the account type in section 4.1, and to read call detail records in section 19.

Provisioning

Six API calls, in sequence, from initial credentials to an agent capable of registering a browser.



7 Step one: mint a customer token

Every iCore call is authorised by a JSON Web Token, and tokens come from one endpoint. The body has exactly three fields, and their names matter: they are `Id`, `Secret` and `Entity`. Not `entity_id`, not `client_id`. The `Entity` field selects which kind of token you get.

POST MINT A CUSTOMER TOKEN

```

POST https://integrationscore.mum1.exotel.com/v2/integrations/token
Content-Type: application/json

{
  "Id":      "c3e57d1c-16f6-40ab-a839-d7f909b070d2",  // Client ID
  "Secret": "<client secret>",
  "Entity": "customer"
}
  
```

200 RESPONSE

```
{
  "RequestId": "ceecabdd-424b-4323-82ea-511697ed9db9",
  "Status": "Success",
  "Code": 200,
  "Error": "",
  "Data": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJJJZCI6..."
}
```

The token is in `Data`, as a bare string. Note the response shape, because every iCore endpoint uses it: a `Status` of "Success" or "Failed", a numeric `Code`, an `Error` string that is empty on success, and a `Data` payload. Check `Status`, not just the HTTP code.

Pass the token in the `Authorization` header exactly as returned, with no `Bearer` prefix. Decoded, its payload is minimal: `{ "Id": "<app or client id>", "exp": 1795881610 }`. There is no account claim inside it, which matters for section 21, item five.

8 Step two: register the application

The application is the object that will own your agents and settings. You create it once, using the customer token, and you hand Exotel your Voice API credentials at the same time so the application can place calls later.

POST REGISTER THE APPLICATION

```
POST https://integrationscore.mum1.exotel.com/v2/integrations/app
Authorization: <customer token>
Content-Type: application/json

{
  "AppName": "webdialer",
  "ExotelAccountSid": "exotel243m",
  "ExotelApiKey": "<api key>",
  "ExotelApiToken": "<api token>",
  "ExotelDomain": "mumbai",
  "IsActive": true
}
```

200 RESPONSE

```
{ "Status": "Success", "Code": 200, "Data": {  
  "AppID": "6d17334a-1684-4eab-b603-94a7d9f855af",  
  "AppSecret": "<returned once, store it now>",  
  
  "CustomerID": "c3e57d1c-16f6-40ab-a839-d7f909b070d2",  
  "AppName": "webdialer",  
  "ExotelAccountSid": "exotel243m",  
  "ExotelDomain": "mumbai",  
  "IsActive": true  
} }
```

MAKE THIS STEP IDEMPOTENT

Registering twice creates a second application, which then competes for the same agents. Before creating, try minting an app token with any stored credentials and calling `GET /v2/integrations/app`. If it returns the application you expect, skip creation entirely. The provisioning script built for this project does exactly that, which is why it can be re-run safely at any time.

9 Step three: mint an application token

Identical endpoint to step one, different entity and different credentials. From here on, every WebRTC call uses this token.

POST MINT AN APPLICATION TOKEN

```
POST https://integrationscore.mum1.exotel.com/v2/integrations/token  
Content-Type: application/json  
  
{  
  "Id": "6d17334a-1684-4eab-b603-94a7d9f855af", // AppID  
  "Secret": "<app secret>",  
  "Entity": "app" // not "customer"  
}
```

The application token measured 164 characters and carried a 90 day expiry on this account. That is long enough that you do not need a refresh loop, but short enough that a long-lived deployment should mint a fresh one per browser session rather than baking one in.

10 Step four: create the WebRTC agent

This is the step a trial account blocks. It creates a user mapping, which is the bridge between an email address you choose and a SIP identity Exotel generates. Note that the request body is an *array*, even for a single agent.

POST CREATE AN AGENT

```
POST https://integrationscore.mum1.exotel.com/v2/integrations/usermapping
Authorization: <app token>
Content-Type: application/json
```

```
[                                     // an array, not an object
  {
    "AppUserId":      "agent@example.com",
    "AppUsername":    "Agent Name",
    "Email":          "agent@example.com",
    "ExotelAccountSid": "exotel1243m",
    "ExotelUserName": "Agent Name",
    "VirtualNumber":  "08048332956", // your ExoPhone
    "AgentNumber":    "8088055789"  // optional fallback phone
  }
]
```

Reading the agent record back returns the values Exotel generated. This record is the primary diagnostic reference for an agent, so each field is described below.

GET READ THE AGENT BACK

```
GET /v2/integrations/usermapping?user_id=agent%40example.com
Authorization: <app token>

{ "Status": "Success", "Data": {
  "AppUserId":      "agent@example.com",
  "ExotelUserId":   "fcd866a88dfc496a94c7d38b606a8395",
  "ExotelAccountSid": "exotel1243m",
  "VirtualNumber":  "08048332956",
  "AgentNumber":    "+918088055789", // normalised to E.164
  "SipId":          "sip:umainihf98a3bc9", // browser SIP username
  "SipSecret":       "<redacted, never expose>",
  "SipDeviceID":    "372299", // the browser device
  "PhoneDeviceID":  "372298", // created by AgentNumber

  "ActiveDeviceId": "372299", // which one actually rings
  "OutboundActive": true,
  "IsActive":       true,
  "Role":           "user"
} }
```

VERIFY ACTIVEDEVICEID AFTER SETTING AGENTNUMBER

Supplying `AgentNumber` makes Exotel create a second device, a phone device, alongside the browser's SIP device. Both exist, but only `ActiveDeviceId` receives calls. Confirm that `ActiveDeviceId` equals `SipDeviceID`, otherwise your calls will be routed to the handset rather than the browser, which presents as a browser fault.

11 Step five: turn on recording

Recording is the field that most people look for in the wrong place. There is no record flag on a WebRTC call. The SDK's call payload is `{customer_id, app_id, to, user_id}` and there is nowhere to put one. Recording is a property of the *application*, and it applies to every call that application places.

GET READ CURRENT SETTINGS

```
GET /v2/integrations/app_setting
Authorization: <app token>

{ "Status": "Success", "Data": [
  { "Key": "record", "Value": "false" } // default
] }
```

POST ENABLE RECORDING FOR EVERY CALL

```
POST /v2/integrations/app_setting
Authorization: <app token>
Content-Type: application/json

{ "Key": "record", "Value": "true" }

// then read it back to confirm
{ "Key": "record", "Value": "true" }
```

Values are strings, not booleans. Send `"true"`, not `true`. Because this is application-wide, there is nothing to remember at call time and nothing a user can forget to tick.

12 Step six: verification

Every failure so far has been a server-side one, and server-side failures are far easier to read than browser ones. Confirm all four of these from a terminal before you open the dialer, because each maps to a specific corresponding browser symptom.

PRE-FLIGHT CHECKS AND THE BROWSER SYMPTOM EACH ONE PREVENTS

Check	Expected	If it fails, the browser shows
POST /token with Entity app	Success	Nothing loads; token request rejected
GET /app	200	SDK initialise returns nothing, silently
GET /app_setting	200	SDK initialise throws before SIP starts
GET /usermapping	Data present	SDK initialise returns nothing, silently

THE APPLICATION SETTINGS CHECK

The SDK fetches application settings during initialisation and treats a non-OK response as fatal. A freshly registered application that has never had a setting written can return 404 here, and the SDK then aborts with only a console warning, returning `undefined` instead of a phone object. The visible symptom is a dialer that never connects, with no error on screen. Writing the recording setting in step five conveniently guarantees this endpoint returns data.

Browser integration

SDK initialisation, SIP registration, and the outbound and inbound call flows.

13 What the SDK does on initialisation

Your code calls two methods. Behind them, the SDK makes three HTTP requests and then opens a WebSocket. Knowing the order lets you pinpoint which provisioning step is wrong when initialisation fails.

BROWSER

```
// token and userId come from your own backend
const sdk = new ExotelCRMWebSDK(accessToken, userId, true);

// third argument true means: auto-register the device
const webPhone = await sdk.Initialize(
  handleCallEvents,    // incoming, connected, callEnded, holdtoggle, mutetoggle
  handleRegistration,  // registered, unregistered, sent_request
  handleSession        // ICE state, media permission diagnostics
);

if (!webPhone) { /* provisioning is wrong. Check the console. */ }
```

The three requests, in order:

1. `GET /v2/integrations/app` resolves which Exotel account this token belongs to.
2. `GET /v2/integrations/app_setting` loads application settings. A 404 here is fatal.
3. `GET /v2/integrations/usermapping?user_id=<id>` fetches the agent, including the SIP credentials.

The SDK then assembles the SIP account details from these responses. The SIP domain is derived from the account SID, which defines the identity model:

INSIDE THE SDK

```
sipdomain: app.ExotelAccountSid + ".voip.exotel.com"
           // becomes "exotel1243m.voip.exotel.com"
userName:  userData.SipId.split(":")[1]
           // "sip:umainihf98a3bc9" becomes "umainihf98a3bc9"
secret:    userData.SipSecret
domain:    "voip.in1.exotel.com:443"
security:  "wss"
```


INITIALIZE RETURNS UNDEFINED INSTEAD OF THROWING

If any of the three requests fails, the SDK logs to the browser console and returns `undefined`. It does not throw, so a `try / catch` will not catch it. Always check the return value. This single behaviour accounts for most "it just does not work and there is no error" reports.

14 SIP registration over WebSocket

With SIP details in hand, the SDK opens a secure WebSocket to `voip.in1.exotel.com:443` and performs a standard SIP registration with digest authentication. The first REGISTER is always rejected; that is the protocol working correctly, not an error.

401 THE EXPECTED CHALLENGE

```
SIP/2.0 401 Unauthorized
Via: SIP/2.0/WSS 34tkmvh8lqns.invalid;rport=63109;branch=z9hG4bK7634239
To: <sip:umainihf98a3bc9@exotel243m.voip.exotel.com>;tag=a1c115...
CSeq: 2 REGISTER
WWW-Authenticate: Digest realm="exotel243m.voip.exotel.com",
                    nonce="aox6zmqMeaL16d/5HnNEI20PkH80cP60", qop="auth"

Server: kamailio (5.7.0 (x86_64/linux))
```

200 REGISTERED

```
REGISTER sip:exotel243m.voip.exotel.com SIP/2.0
Authorization: Digest algorithm=MD5, username="umainihf98a3bc9",
                realm="exotel243m.voip.exotel.com", response="97a3d8ed...",
                qop=auth, cnonce="i54qprsljaej", nc=00000001
Contact: <sip:ej9v5cl9@34tkmvh8lqns.invalid;transport=wss>;expires=300

SIP/2.0 200 OK
Contact: <sip:ej9v5cl9@49.37.163.125:63109;transport=wss>;expires=300
```

Your registration callback now fires with `"registered"`. The registration lasts 300 seconds and the SDK refreshes it. Three states are emitted in practice: `sent_request` while in flight, `registered` on success, and `unregistered` on failure or when the transport drops. Treat `unregistered` as temporary and re-register rather than tearing everything down.

15 Placing an outbound call

Exotel places outgoing calls on your behalf rather than dialling directly from the browser. This section describes that flow and the handling it requires.

`webPhone.MakeCall(number, callback)` asks Exotel to place a call. Exotel then brings you into it by sending a SIP INVITE to the browser you registered earlier, and joins that leg to the customer. So an outgoing call arrives back at your event handler as an `"incoming"` event.

This design has two practical advantages. Because Exotel originates both legs, caller ID, routing, recording, billing and call detail records are managed centrally, and the browser requires no knowledge of the telephone network. It also means a single event path handles both inbound and outbound calls, so media is connected in one place in your application. The requirement it places on your code is to answer the leg Exotel offers.

POST THE REQUEST MAKECALL ISSUES

```
POST https://integrationscore.mum1.exotel.com/v2/integrations/call/outbound_call
Authorization: <app token>
Content-Type: application/json

{
  "customer_id": "c3e57d1c-16f6-40ab-a839-d7f909b070d2",
  "app_id":      "6d17334a-1684-4eab-b603-94a7d9f855af",
  "to":         "8088055789",
  "user_id":    "agent@example.com"
}
// note: no field for caller ID, and none for recording
```

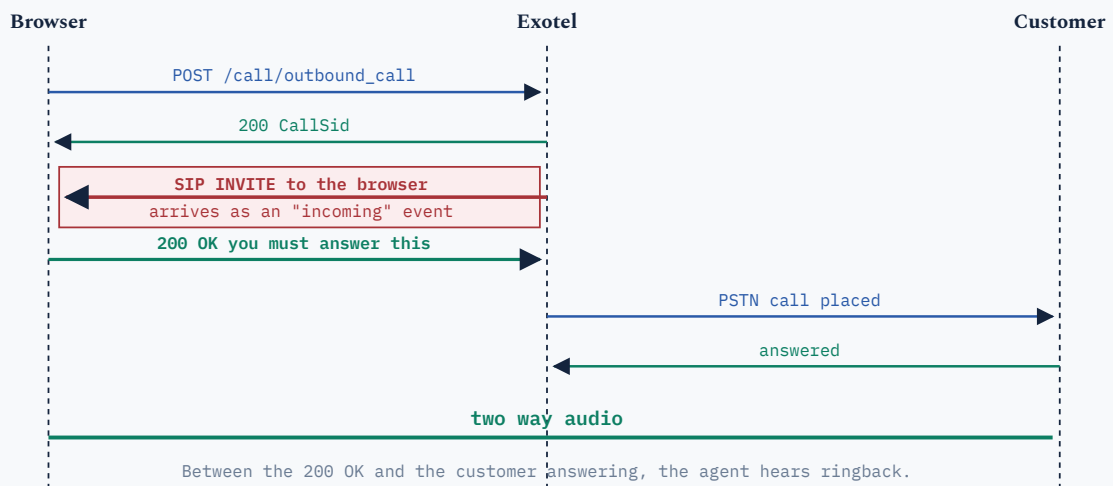


Figure 3. The complete outbound call sequence. The highlighted arrow is the leg that requires handling: Exotel invites the browser into the call the agent requested. Answering it automatically presents the agent with a normal outgoing call.

The practical consequence: you must track that you initiated a call, and auto-answer the INVITE that comes back. Roughly this shape:

HANDLING THE RETURN LEG

```
function dial(number) {
  state.pendingOutbound = true; // set BEFORE calling
  webPhone.MakeCall(number, (status, response) => {
    if (status === "success") log(response.Data.CallSid);
  });
}

function handleCallEvents(event, data) {
  if (event === "incoming") {
    if (state.pendingOutbound) { // our own call coming back
      webPhone.AcceptCall(); // answer silently

      return;
    }
    showIncomingPopup(data.callFromNumber); // a genuine inbound call
  }
}
```

Set the flag before calling `MakeCall`, not inside its success callback. The INVITE can reach the browser before the HTTP response does, and if the flag is not set yet you will show a spurious incoming popup.

HANDLING A CANCELLED DIAL

If the agent hangs up while still dialling, there is no SIP session to end yet, so `HangupCall` is a no-op and Exotel's INVITE may still arrive seconds later. Record a short-lived "abandoned" timestamp and refuse any INVITE that arrives inside that window, otherwise a cancelled call reappears as a mystery inbound ring.

16 Receiving an inbound call

A genuine inbound call is simpler, because there is no HTTP request involved at all. A customer dials the ExoPhone, the dashboard flow routes the call to the VoIP agent, and Exotel sends an INVITE to whichever browser currently holds the registration. Your `"incoming"` handler fires with the caller's number in `callFromNumber`, you ring and show a popup, and `AcceptCall()` answers.

The INVITE carries useful headers that the SDK stores for you, including `X-Exotel-CallSid`, which is the same CallSid you would use to look the call up later.

ONE AGENT MEANS ONE REGISTRATION

An agent is a single SIP identity, and a SIP identity can hold one registration. Open the same agent in two browsers and both register the same identity, so Exotel rings whichever registered most recently. The resulting failure is difficult to diagnose: calls are delivered to an unattended machine while the intended agent, whose dialer reports a healthy registration, receives nothing. Give every concurrent user their own agent record, and treat "provision one agent per seat" as the default rather than an optimisation.

17 Microphone access and browser policy

Three browser rules apply, and all three are non-negotiable from your code.

Secure origin

`getUserMedia` is unavailable outside a secure context. In practice that means `https://`, or `localhost` which browsers treat as secure by exception. Serving the dialer over plain HTTP on a LAN address will fail with no microphone at all, and the failure looks like a broken app rather than a policy decision.

Permission requires a user gesture

Browsers show the microphone prompt in response to a user action. Ask for it deliberately, on a button, rather than letting the SDK trigger it in the middle of the first call. Query the current state first so you can skip the prompt when it is already granted:

ASKING WELL

```
const perm = await navigator.permissions.query({ name: "microphone" });

if (perm.state === "granted") { /* open and release, silently */ }
if (perm.state === "prompt") { /* show your own button first */ }
if (perm.state === "denied") { /* explain the padlock icon */ }
```

On macOS there is a second gate above the browser. Chrome itself must be allowed the microphone in System Settings, Privacy and Security. If the operating system is blocking it, Chrome's own prompt never appears and the page simply reports that permission was denied.

Audio playback requires the same gesture

The same gesture that grants the microphone is what unlocks audio playback. If you create an `AudioContext` for ringtones at page load it starts suspended and your ringer is silent. Create or resume it inside the permission click. The SDK ships its own ringtone files and plays them on detached audio elements, which are subject to exactly the same policy.

Operations

Call recording, call detail records, and production deployment requirements.

18 Call recording

Covered mechanically in section 11. Three operational points are worth stating separately.

First, it is all or nothing per application. If you need recording on some calls and not others, you need two applications, and agents belong to one application each. Second, the setting takes effect immediately for calls placed after it is written; it does not retroactively record anything. Third, recordings surface against the call in the Exotel dashboard and via the call detail record described next.

RECORDING AND CONSENT

Jurisdictions differ on whether one party or all parties must consent to a recorded call. Because this setting is account-wide and silent, nothing in the product will remind an agent that a call is being recorded. If you need an announcement or a consent prompt, that belongs in your call flow or your agent script, and it is worth deciding before you switch this on.

19 Reading call detail records

The Voice API returns the authoritative record of what happened. This is the fastest way to settle whether a problem was in the browser or on the network, because it shows what Exotel believes occurred.

GET A CALL RECORD

```
GET https://api.in.exotel.com/v1/Accounts/exotel243m/Calls/<CallSid>.json
Authorization: Basic base64(API_KEY:API_TOKEN)
```

```
{ "Call": {
  "Sid": "0c423e88bb65c8f28280157419401a8p",
  "From": "sip:umainihf98a3bc9", // the browser leg
  "To": "08088055789",
  "PhoneNumberSid": "08048332956", // caller ID shown
  "Status": "completed",
  "StartTime": "2026-08-25 18:20:09",
  "EndTime": "2026-08-25 18:20:28",
  "Duration": 19,
  "Direction": "outbound-dial"
} }
```

A `From` value beginning `sip:` confirms the call originated from a browser rather than a handset, which is a quick way to confirm that the WebRTC path was used.

DO NOT PUT CREDENTIALS IN THE URL

Exotel's documentation often shows Voice API calls in the form `https://key:token@api.in.exotel.com/...`. Modern HTTP clients reject that. Node's built-in `fetch` throws outright: *"Request cannot be constructed from a URL that includes credentials."* Use an `Authorization: Basic` header instead. Worth checking early: the request never reaches Exotel, so there is nothing on the platform side to look at.

20 Production deployment

HTTPS is required

Because `getUserMedia` requires a secure context, the moment you want to use the dialer from a machine other than the one running the server, you need HTTPS. Any platform that terminates TLS for you will do. If you deploy behind a proxy, remember to trust it, or your framework will report the request as insecure and cookies marked Secure will never be set.

Protect the token endpoint

An endpoint that mints application tokens is, functionally, an endpoint that lets the caller place calls billed to your account and read your agents' SIP credentials. On localhost the exposure is limited to the local machine; on a public URL it extends to anyone who can reach it. Place authentication in front of this endpoint before the first deployment.

RECOMMENDED MINIMUM

A shared password, a signed session cookie with an expiry, constant-time comparison, and per-address backoff on failed attempts is enough for a small internal tool and takes well under an hour. Leave your status callback route open, since the telephony platform posts to it and it carries no secrets.

Run provisioning locally

Provisioning writes the application credentials back into your environment file. A deployed container's filesystem is discarded on redeploy, so run provisioning from a workstation and copy the resulting values into your platform's environment variables.

Implementation notes

Five implementation details, each with the observed behaviour, its cause, and the recommended handling.

21 Five implementation considerations

01 Outgoing calls appear as incoming calls

- SYMPTOM** Agent dials a number and immediately gets an incoming call popup for it.
- CAUSE** `MakeCall` only asks Exotel to place the call. Exotel then sends a SIP INVITE to the browser, which surfaces as an `"incoming"` event.
- FIX** Track a `pendingOutbound` flag, set before calling `MakeCall`, and auto-answer the matching INVITE. See section 15.

02 Every call failure reports the word "Error"

- SYMPTOM** All failures show as the literal string `Error`, with no reason anywhere in the UI.
- CAUSE** The SDK reports failures as `new Error(response.statusText)`. Exotel serves over HTTP/2, where `statusText` is always an empty string. The real reason is in the response body, which the SDK discards.
- FIX** Wrap `window.fetch`, and on a failed request to `/call/outbound_call` read the cloned body and extract `error_data.description`.

03 A plus sign in the agent email breaks initialisation

- SYMPTOM** An agent such as `name+web rtc@gmail.com` provisions correctly on the server but the SDK never initialises.
- CAUSE** The SDK interpolates the user id into a query string without encoding it. In a query string `+` means space, so Exotel looks up `name web rtc@gmail.com`, returns 404, and initialisation aborts.
- FIX** Pass `encodeURIComponent(userId)` into the SDK constructor. That value is used only to build this URL; the SDK builds its user object from the response. The raw form returns 404; the encoded form returns 200.

04 Setting an agent phone number diverts calls to a handset

- SYMPTOM** After adding `AgentNumber`, calls ring a mobile rather than the browser.
- CAUSE** Exotel creates a second device, a phone device, and `ActiveDeviceId` decides which one rings.
- FIX** After provisioning, assert `ActiveDeviceId === SipDeviceID`. Leave `AgentNumber` empty for browser-only agents.

05 Account identity comes from the token, not from your request

- SYMPTOM** A request configured for one account resolves to a different account name.
- CAUSE** The SDK derives the account entirely from `GET /v2/integrations/app`, which reflects the token. The `ExotelAccountSid` you send in provisioning bodies is never checked against it, and the SIP realm is then built from whatever the token resolved to.
- FIX** After minting a token, call `GET /app`, read the authoritative `ExotelAccountSid`, and reject the request if it is not the account you expect. Treat any account SID supplied by a client as untrusted input.

ACCOUNT LEVEL ISSUES

The five items above are all things you handle in your own application. Anything that lives on the account rather than in your code, whether that is WebRTC enablement, balance and billing, number provisioning, or how an ExoPhone routes an inbound call, is quicker to settle with Exotel than to infer from an error message. Email `hello@exotel.in` with your Account SID and, where relevant, the CallSid, and they can see the state of the account directly.

A useful rule while debugging: if SIP registration is healthy and the failure only appears at call time, look at the account before you look at your code.

22 Error reference

ERRORS OBSERVED LIVE, AND WHAT EACH ONE MEANS

Code	Message	Meaning and action
403	This is a trial account. Operation not permitted.	Agent creation is enabled by Exotel. Email <code>hello@exotel.in</code> for IP-PSTN WebRTC enablement, using the template in section 5.1.
402	Insufficient balance to make a call. Please recharge your account. (10720)	Nothing is wrong with your integration. Top up the account, or ask Exotel support to confirm the balance and billing state.
404	record not found	User mapping lookup failed. Either the agent is not provisioned, or the id was not URL encoded. See item three.
404	No matching results	Voice API could not find that CallSid. Usually a typo. Importantly, this proves authentication succeeded.
401	SIP/2.0 401 Unauthorized	Not an error. The expected digest challenge during SIP registration. A second REGISTER follows.
n/a	Initialize() returned undefined	One of the three initialisation requests failed. Check the console, then re-run the four pre-flight checks in section 12.
n/a	Request cannot be constructed from a URL that includes credentials	Your HTTP client rejecting credentials in a URL. Move them to an Authorization header, as in section 19.

23 Verification checklist

Work down the list. Each item depends on the ones above it, so the first failure is the real problem and everything below it is noise.

- ☐ **Account is Full, active, KYC complete.** `GET /v1/Accounts/<sid>.json`
- ☐ **Account has balance.** Prepaid accounts fail at call time only.
- ☐ **Customer token mints.** `POST /token` with `Entity: customer`
- ☐ **Application exists.** `GET /app` returns your AppID and the right account SID
- ☐ **App token mints.** `POST /token` with `Entity: app`
- ☐ **Settings endpoint answers.** `GET /app_setting` returns 200, not 404
- ☐ **Recording is on.** `record` is the string `"true"`
- ☐ **Agent exists.** `GET /usermapping` returns Data, with the id URL encoded
- ☐ **Browser is the active device.** `ActiveDeviceId === SipDeviceID`

☐ **Page is on a secure origin.** HTTPS, or localhost

☐ **Microphone granted.** And on macOS, granted to the browser by the OS too

☐ **SIP registers.** REGISTER, 401, REGISTER, 200 OK

☐ **Outgoing call connects.** And the return INVITE is auto-answered

☐ **Call record confirms it.** begins with

☐ **Inbound flow is configured.** Dashboard, ExoPhone to VoIP agent

☐ **Token endpoint is authenticated.** Before the first public deploy

Reference

Consolidated endpoint reference and glossary.

A Endpoint reference

Base for iCore is `https://integrationscore.mum1.exotel.com`. Base for Voice is `https://api.in.exotel.com`. Region strings differ for other Exotel regions.

ICORE INTEGRATIONS

Method	Path	Auth	Purpose
POST	/v2/integrations/token	none	Mint a JWT. Body selects customer or app entity.
POST	/v2/integrations/app	customer	Register an application. Returns AppID and AppSecret.
GET	/v2/integrations/app	app	Resolve which account this token belongs to.
POST	/v2/integrations/usermapping	app	Create agents. Body is an array.
GET	/v2/integrations/usermapping? user_id=	app	Read an agent. URL encode the id.
GET	/v2/integrations/app_setting	app	Read application settings.
POST	/v2/integrations/app_setting	app	Write one setting as a Key and Value pair.
POST	/v2/integrations/call/outbound_call	app	Place a call. Exotel then INVITEs the browser.

VOICE API V1, HTTP BASIC AUTH

Method	Path	Purpose
GET	/v1/Accounts/<sid>.json	Account type, status, billing type, KYC status.
GET	/v1/Accounts/<sid>/Calls/<CallSid>.json	Call detail record: status, duration, direction, price.
POST	/v1/Accounts/<sid>/Calls/connect.json	Click to call, connecting two ordinary phone numbers.

B Glossary

API	A way for two computer systems to talk. Instead of clicking buttons on a website, one program asks another program to do something.
Token	A temporary pass. Rather than sending your password every time, you exchange it once for a pass that expires.
Provisioning	Setting up an account so it is ready to use. Here, telling Exotel that a particular person is allowed to make calls from a browser.

Agent	The person taking or making calls, and also the record representing them in Exotel.
ExoPhone	Your business phone number, provided by Exotel. Customers see it and can ring it.
Registration	The browser telling Exotel "I am here and ready for calls". If it lapses, calls cannot reach you.
Inbound and outbound	Calls coming in to you, and calls going out from you.
Prepaid balance	Money loaded onto the Exotel account in advance. When it runs out, calls stop.
HTTPS	The secure version of a web address. Browsers refuse to hand over a microphone to a page that is not on HTTPS.

Every request and response body in this document was captured from a live Exotel account during a working integration. Secrets, SIP passwords and application secrets have been replaced with placeholders. Account identifiers and call identifiers are retained so the shapes are realistic.